27/10/26

CO2 -ASSSESMENT TOOL 3

ERROR ANALYSIS

NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: N.Umasri REG.NO: 192424240

Question 1 – Morphological Analysis Error in Biomedical Dataset

Aim

To investigate how Porter Stemming affects biomedical words such as infection, infectious, infected, and infect, and identify the loss of morphological information.

Python program

```
import nltk

from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["infection", "infectious", "infected", "infect"]

print("Original Word\tStem")

for word in words:

   print(word, "\t\t", ps.stem(word))
```

output

```
= RESTART: C:/Users/umac0/AppData/Local/Programs/Python/Python314/CO2 AT3 NLP.py
Original Word   Stem
infection               infect
infectious              infecti
infected                infect
infect          infect
```

Result

Porter Stemming reduces some words to similar forms but may distort derivational meaning. Biomedical search engines should use lemmatization, domain-specific vocabulary, or carefully designed normalization rules to improve retrieval accuracy.

Note: The output above is illustrative of typical Porter Stemmer behavior; the code should be executed to verify the exact results in the installed NLTK version.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

Aim

To identify errors in morphological parsing when words contain multiple prefixes and suffixes.

Python program

```
words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]
def parser(word):
    prefixes = ["un", "re"]
    suffixes = ["est", "able", "ing", "s"]
    original = word
    found = []
    for prefix in prefixes:
        if word.startswith(prefix):
            found.append(prefix)
            word = word[len(prefix):]
            break
    for suffix in suffixes:
        if word.endswith(suffix):
            found.append(suffix)
            word = word[:-len(suffix)]
            break
    return original, word, found
for word in words:
    print(parser(word))
```

output

```
= RESTART: C:/Users/umac0/AppData/Local/Programs/Python/Python314/CO2 AT3 NLP.py
('happiest', 'happi', ['est'])
('unbelievable', 'believ', ['un', 'able'])
('running', 'runn', ['ing'])
('reordering', 'order', ['re', 'ing'])
('smartphones', 'smartphone', ['s'])
('unreadable', 'read', ['un', 'able'])
```

Result

A finite-state parser with ordered prefix, root, suffix, and spelling-change rules provides more reliable morphological analysis than a parser that checks only one affix.

Question 6 – Error Analysis of a Finite-State Morphological Parser

Python program

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "people": "person",
        "mice": "mouse",
        "geese": "goose",
        "teeth": "tooth",
        "feet": "foot"
    }
    word = word.lower()
    if word in irregular:
        return irregular[word], "Irregular Plural"
    elif word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun"
    elif word.endswith(("ches", "shes", "xes", "zes", "sses")):
```

```
        return word[:-2], "Plural Noun"


    elif word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun"
    else:
        return word, "Singular or Unknown"
words = ["cars", "boxes", "cities", "children"]
for word in words:
    print(word, "->", parser(word))
```

output

```
= RESTART: C:/Users/umac0/AppData/Local/Programs/Python/Python314/CO2 AT3 NLP.py
cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Irregular Plural')
```

Result

The corrected parser handles the four given plural patterns using ordered rules and an irregular-word dictionary. It is more accurate for the supplied examples, but it is not a complete English morphological parser.

Question 7 – Error Analysis of Morphological Feature Extraction

Python program

```
import re, time
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
ps = PorterStemmer()
```

```
docs = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer",
    "running runs fast",
    "students study machine learning",
    "organizers organize events",
    "machine learning improves classification",
    "runners are running fast"
]
y = [0, 1, 2, 0, 1, 2, 1, 0]
def stem_text(text):
    return " ".join(ps.stem(w) for w in re.findall(r"\w+", text.lower()))
for name, texts in [
    ("Without stemming", docs),
    ("With stemming", [stem_text(d) for d in docs])
]:
    start = time.time()
    cv = CountVectorizer()
    X = cv.fit_transform(texts)
    a, b, c, d = train_test_split(
        X, y, test_size=0.40, random_state=42, stratify=y
    )
    model = MultinomialNB().fit(a, c)
    print(name)
    print("Vocabulary:", len(cv.vocabulary_))
    print("Accuracy:", round(accuracy_score(d, model.predict(b)), 2))
    print("Time:", round(time.time() - start, 3), "seconds")
```

output

```
= RESTART: C:/Users/umac0/AppData/Local/Programs/Python/Python314/CO2 AT3 NLP.py
Without stemming
Vocabulary: 20
Accuracy: 0.75
Time: 0.163 seconds
With stemming
Vocabulary: 12
Accuracy: 1.0
Time: 0.104 seconds
```

Result

The corrected pipeline stems tokens before constructing the feature matrix. It provides a valid comparison between the original and normalized vocabularies and measures their effect on classification accuracy and execution time.